Pongo Inventory OS

Detailed Project Scope and Build Requirements

Prepared from the project scope discussed in chat. No build has started yet.

Area	Scope
Project owner	Pongo Pet Supplies
Document purpose	Consolidate every requirement discussed so far into one scope document for review before starting Codex work.
Target product	Standalone inventory, fulfillment, receiving, reporting, and route operations system.
Current decision	Do not build as a WordPress plugin. Build as a standalone service that connects to WooCommerce through API.
Prepared date	2026-07-07

Contents

1. Executive Summary

Pongo Inventory OS is planned as a standalone internal operations system for Pongo Pet Supplies. The goal is to replace the parts of Zenventory that Pongo actually uses, while avoiding unnecessary enterprise

warehouse complexity. The system will sync product, variation, stock, order, customer, shipping, and billing data from WooCommerce through the WooCommerce REST API. It will store Pongo-specific operational data such as barcode, manufacturer, location, unit cost, par level, received inventory, allocated stock, sellable stock, cycle counts, CSV reports, and delivery routes in its own database.

The first major build target should be the Items module. That module becomes the foundation for receiving, locations, cycle count, order allocation, picking, reporting, and routing.

2. Core Product Direction

Area	Scope
Product name	Pongo Inventory OS
Product type	Standalone web application, not a WordPress plugin.
Primary goal	Replace Zenventory for Pongo-specific inventory operations.
Deployment target	Heroku or a similar cloud platform later.
Integration source	WooCommerce REST API for products, variations, stock, prices, orders, customers, addresses, and line items.
Operational source	Pongo Inventory OS owns operational inventory data such as barcode, unit cost, locations, allocations, receipts, cycle counts, and reports.
Critical rule	Every simple WooCommerce product and every WooCommerce variation must become its own individual inventory item in Pongo Inventory OS.

3. In Scope and Out of Scope

3.1 In Scope

- Standalone FastAPI plus React web app with PostgreSQL database.
- Items module with Zenventory-like admin interface.
- WooCommerce product and variation sync using a Refresh button.
- Remap function to relink local items to WooCommerce products or variations.
- Every simple product and every variation as a separate inventory item.
- Bulk product import and bulk product export using CSV.
- Manual product and item creation and editing.
- Barcode, manufacturer, manufacturer website, brand, cost, price, stock, and location management.
- Preset inventory locations imported from uploaded location data.
- Stock allocation by location.
- Inventory export and inventory export broken down by location.
- Direct receiving without purchase orders.
- Received inventory report by date range.
- Cycle count using SKU or barcode scanning.

- Open Orders, Allocate Orders, and Pick Orders only.
- Order fulfillment export and SKU or barcode order report.
- Separate route creation and route optimization feature on map.
- Audit or stock movement records for every stock-changing action.

3.2 Out of Scope for Now

- Supplier management.
- Purchase order creation and purchase order receiving.
- Supplier quote portal.
- Complex receiving workflow with PO dependencies.
- Put-away workflow as a separate module.
- Complex order stages such as picking started, packed, ready for delivery, out for delivery, delivered, and returned.
- Generic enterprise warehouse features not needed for Pongo right now.
- EOQ or ABC forecasting as an initial feature.
- Building inside WordPress as a plugin or shortcode app.

4. Architecture and Tech Stack

Area	Scope
Backend	FastAPI
Database	PostgreSQL
ORM	SQLAlchemy
Migrations	Alembic
Frontend	React
Deployment	Heroku later, with local development first.
WooCommerce integration	Backend-only WooCommerce REST API client.
Authentication	Internal staff login, session or JWT. Exact auth choice to be decided during setup.
CSV processing	Backend-driven import/export jobs with validation, preview, import errors, and failed-row downloads.
Map and routing	Provider abstraction for Google Maps, Google Routes, Mapbox, or OpenRouteService. Keys stay backend-side.

```
Recommended repository structure:

pongo-inventory-os/
  backend/
    app/
      api/
      core/
      db/
      models/
      schemas/
      services/
    alembic/
    tests/
    requirements.txt
    .env.example
```

```
frontend/
  src/
    api/
    components/
    features/
    pages/
    types/
  package.json
docs/
  PRD.md
  DATABASE_SCHEMA.md
  API_SPEC.md
  CSV_COLUMNS.md
  WOOCOMMERCE_SYNC.md
  UI_REFERENCE.md
  BUILD_PLAN.md
legacy-wordpress-plugin/
README.md
```

5. WooCommerce Integration

WooCommerce remains the customer-facing storefront. Pongo Inventory OS becomes the operational inventory layer.

Area	Scope
Sync flow	WooCommerce API -> backend sync service -> Pongo Inventory OS database -> frontend UI.
Do not do	Do not read live from WooCommerce on every page load.
Credentials	Store WOOCOMMERCE_BASE_URL, WOOCOMMERCE_CONSUMER_KEY, and WOOCOMMERCE_CONSUMER_SECRET only in backend environment variables.
First sync mode	Read-only safe mode first. Do not update WooCommerce stock until local behavior is tested.

5.1 Data Pulled from WooCommerce

- Products and variations.
- SKUs.
- Product names and variation names.
- Descriptions.
- Categories.
- Images.
- Regular price, sale price, and sales price.
- Weight and dimensions.
- Stock quantity and stock status.
- Product ID and variation ID.
- Orders and order statuses.
- Customers, shipping address, billing address, phone, email, company, and line items.

5.2 Data Owned by Pongo Inventory OS

- Barcode.
- Manufacturer and manufacturer website.
- Client.
- Warehouse.
- Inventory location and default location.
- Allocated stock.
- Sellable stock.
- Under par status.
- On order quantity.
- Unit of measurement.
- Recommended retail price.
- Unit cost.
- Default economic order quantity.
- Default lead time days.
- Par level.
- Assembly flag.
- Serializable flag.
- Track lot flag.
- Perishable flag.
- Re-order flag.
- Storage volume.
- Brand override when not available from WooCommerce.
- Receipt, cycle count, allocation, pick, and route records.

6. Items Module

The Items module is the first real screen and the foundation for the entire system. It should be inspired by the Zenventory Admin -> Items page shown in the uploaded video, but built with Pongo branding and standalone architecture.

6.1 Items Page Layout

- Page title: Items.
- Tabs: Main, Nutrition, Categories, Customizations. Main is required first; others can be placeholders.
- Search input.
- Category filter.
- Active and inactive filter.
- Include Non Inventory checkbox.
- Reset button.
- Clear button.
- Refresh button.
- Remap button.
- Export button.
- Items table.

- Edit icon or edit drawer.
- Product image.
- Row actions dropdown.

6.2 Items Table Columns

Required Items Table Columns

Image	SKU	Description
Category	UOM	Unit Cost
Sales Price	Recommended Retail Price	Barcode
Brand	Manufacturer	Manufacturer Website
Warehouse	Inventory Location	Default Location
In Stock	Allocated	Sellable
Under Par	On Order	Weight
Active Status		

6.3 Refresh Button Behavior

- Fetch latest products and variations from WooCommerce.
- Create one local inventory item for each simple product.
- Create one local inventory item for each variation.
- Update WooCommerce-owned fields locally.
- Preserve OS-owned fields such as barcode, manufacturer, manufacturer website, unit cost, warehouse, location, par level, reorder settings, brand overrides, and custom metadata.
- Return created count, updated count, skipped count, error count, and errors.

6.4 Remap Button Behavior

- Used when a local item is not correctly linked to WooCommerce.
- Allow search and mapping by Woo product ID, Woo variation ID, SKU, barcode, or product name.
- Save woo_product_id and woo_variation_id on the local item.
- Pull WooCommerce fields again while preserving OS-owned fields.
- Log the remap event.

7. Inventory Data Model and Columns

The master inventory item/export structure must support these columns. WooCommerce will fill what it can. The rest will be filled manually or through CSV import.

Master Inventory Columns

Client	SKU	Description
Category	Unit of Measurement	Warehouse
In Stock	Allocated	Sellable
Under Par	On Order	Barcode
Manufacturer Website	Recommended Retail Price	Sales Price
Unit Cost	Weight	Default Econ Order
Default Lead Time (Days)	Par Level	Assembly

Serializable	Track Lot	Perishable
Re-Order	Storage Length	Storage Width
Storage Height	Storage Volume	Brand

7.1 Calculated Fields

Area	Scope
Sellable	In Stock minus Allocated.
Under Par	True when In Stock is less than or equal to Par Level.
Storage Volume	Storage Length times Storage Width times Storage Height.
Inventory Value	In Stock times Unit Cost.
Unit Cost Total	Quantity times Unit Cost.
Total Price	Quantity times Unit Price.

8. Locations and Stock by Location

Locations are a core feature. Pongo will upload existing location data, and the system should preset those locations. Stock should be assignable to warehouse locations.

- A SKU can exist in multiple physical locations.
- A product can have one default location but can also have stock in other locations.
- Total stock should be the sum of stock across item-location rows.
- Total allocated should be the sum of allocated across item-location rows.
- Total sellable should be the sum of sellable across item-location rows.
- Receiving must increase stock for a specific item-location row.

8.1 Location-Based Inventory Export Columns

Inventory Export by Location Columns

Client	SKU	Description
Category	Unit of Measurement	Warehouse
Inventory Location	Default Location	In Stock
Allocated	Sellable	Under Par
On Order	Barcode	Manufacturer Website
Recommended Retail Price	Sales Price	Unit Cost
Weight	Default Econ Order	Default Lead Time (Days)
Par Level	Assembly	Serializable
Track Lot	Perishable	Re-Order
Storage Length	Storage Width	Storage Height
Storage Volume	Brand	

9. Bulk Import, Bulk Export, and Manual Item Management

9.1 Bulk Import

- Upload CSV using the column structure Pongo provides.
- Validate required fields.
- Preview rows before final import.
- Create new items when SKU or barcode does not exist.
- Update existing items when SKU or barcode already exists.
- Track import job history.
- Show successful row count and failed row count.
- Store import errors with row number and raw row.
- Allow failed rows to be downloaded as CSV.
- Do not overwrite WooCommerce-owned fields unless explicitly supported by backend logic.

9.2 Bulk Export

- Export all inventory columns.
- Filters: all items, in-stock only, out-of-stock only, under-par only, category, brand, warehouse, client, location.
- Support both summary export and location-breakdown export.

9.3 Manual Create and Edit

- Create local inventory item manually.
- Edit existing inventory item.
- Search by SKU, barcode, or name.
- Edit barcode, manufacturer, manufacturer website, unit cost, recommended retail price, sales price, brand, category, UOM, warehouse, location, par level, reorder fields, perishable, track lot, serializable, dimensions, weight, and active status.
- For MVP, manual item creation can be local-only. Push-to-WooCommerce can be added later.

10. Direct Receiving Without PO

Pongo does not need purchase order receiving right now. The core receiving workflow is direct receiving without PO.

- Scan barcode or enter SKU.
- Find item.
- Enter quantity received.
- Enter unit cost.
- Select warehouse.
- Select inventory location.
- Optionally enter lot number, expiration date, PKG #, item #, pallet #, sales price, weight, and notes.
- Submit receiving.

- Increase stock for that item-location row.
- Update total item stock.
- Recalculate sellable and under par.
- Create receipt and receipt item rows.
- Create stock movement or audit row.
- Update WooCommerce stock through API only when safe and configured.

10.1 Bulk Receiving Session

- Scan or search multiple items.
- Each row has SKU or barcode, quantity received, unit cost, warehouse, location, and notes.
- Submit all rows together as one receipt session.
- Generate one receipt number for the session.
- Create one receipt item and one stock movement per row.

11. Reports

Required reports are inventory export, inventory export by location, received inventory report, fulfillment details export, SKU/barcode order report, and CSV exports for all reports.

Received Inventory Report Columns

Receipt	SKU	Category
Description	Quantity	UOM
Unit Cost Total	Quantity (Base UOM)	Lot Number
Expiration Date	PKG #	Item #
Pallet #	Unit Cost	Sales Price
Weight	Brand	Client
Received Date	Warehouse	PO# / Receipt#
Name		

11.1 Received Inventory Report Behavior

- Show items received within a selected date range.
- Since Pongo does not use POs, generate a receipt number such as RCPT-2026-00045.
- Use the receipt number for both Receipt and PO# / Receipt# when no PO exists.
- Filters: start date, end date, SKU, barcode, category, brand, warehouse, inventory location, client.

Order Fulfillment Export Columns

CO#	Client	SKU
Ordered Qty	Ordered UOM	Completed On
Unit Cost	Unit Cost Total	Placed On
On Hold	Hold Until	Unit Price
Total Price	Description	Line #
Customer	Shipping Address 1	Shipping Address 2
Shipping Address 3	Shipping City	Shipping State
Shipping Country	Shipping Zip	Shipping Phone
Billing Address 1	Billing Address 2	Billing Address 3

Billing City	Billing State	Billing Country
Billing Zip	Billing Phone	Created By
Brand	Tracking Number	Company

11.2 SKU or Barcode Order Report

- Search by SKU, barcode, product name, and date range.
- Show every order that included the SKU or barcode.
- Purpose: see quantity sold, related orders, customer details, revenue, cost, and margin.

Suggested SKU/Barcode Order Report Columns

SKU	Barcode	Description
Brand	Order Number	Order Date
Customer	Ordered Qty	Unit Cost
Unit Price	Unit Cost Total	Total Price
Order Status	Completed On	

12. Order Workflow

Only three stages are needed: Open Orders, Allocate Orders, and Pick Orders.

Area	Scope
Open Orders	Pull eligible WooCommerce orders. Show order number, customer, placed date, status, total items, total quantity, allocation status, and action to allocate.
Allocate Orders	Match order items to inventory items by Woo product ID, variation ID, SKU, or barcode. Check sellable stock. Allocate if enough stock exists. Show shortage if not enough stock exists. Increase allocated quantity and record allocation.
Pick Orders	Staff opens allocated order, scans barcode or enters SKU, system matches item to order line, increments picked quantity, shows ordered/allocated/picked quantities, prevents overpicking, and allows completion when all items are picked.
Complete Order	When picked, update local order status and update WooCommerce order status to completed through API. Record fulfillment data.

13. Cycle Count

- Cycle count is a must-have module.
- Scan barcode or enter SKU.
- Show item and current stock.

- Enter counted stock.
- Calculate difference.
- If difference is not zero, require a reason.
- Submit updates stock.
- Create stock movement or audit row.
- Update WooCommerce stock only if configured and safe.

14. Route Creation and Optimization

Route creation and route optimization are required as a separate module, not mixed into receiving or picking.

- Select route date.
- Pull eligible WooCommerce orders.
- Select orders for route.
- Use shipping addresses as stops.
- Show stops on map.
- Optimize stop sequence.
- Save route.
- Export route later.
- Use provider abstraction so the route engine can later use Google Maps, Google Routes, Mapbox, or OpenRouteService.
- Never expose map API keys in frontend. Store keys in backend environment variables.

15. Database Model Draft

inventory_items

id	client	woo_product_id	woo_variation_id
sku	barcode	description	category
unit_of_measurement	warehouse	in_stock	allocated
sellable	under_par	on_order	manufacturer
manufacturer_website	recommended_retail_price	sales_price	unit_cost
weight	default_econ_order	default_lead_time_days	par_level
assembly	serializable	track_lot	perishable
reorder	storage_length	storage_width	storage_height
storage_volume	brand	image_url	active
source	created_at	updated_at	

inventory_locations

id	client	warehouse	location_code
location_name	zone	aisle	rack
shelf	bin	is_default	active
created_at	updated_at		

inventory_item_locations

id	inventory_item_id	location_id	warehouse
inventory_location	is_default_location	in_stock	allocated
sellable	on_order	created_at	updated_at

stock_movements

id	inventory_item_id	inventory_location_id	sku
barcode	movement_type	quantity_change	old_stock
new_stock	unit_cost	reason	reference_type
reference_id	created_by	created_at	

receipts

id	receipt_number	client	warehouse
received_by	received_date	notes	created_at

receipt_items

id	receipt_id	inventory_item_id	inventory_location_id
sku	category	description	quantity
uom	quantity_base_uom	unit_cost	unit_cost_total
sales_price	weight	brand	client
lot_number	expiration_date	pkg_number	item_number
pallet_number	warehouse	received_date	po_or_receipt_number
name	created_at		

orders

id	woo_order_id	order_number	customer_name
customer_email	placed_on	completed_on	status
allocation_status	shipping_address_1	shipping_address_2	shipping_address_3
shipping_city	shipping_state	shipping_country	shipping_zip
shipping_phone	billing_address_1	billing_address_2	billing_address_3
billing_city	billing_state	billing_country	billing_zip
billing_phone	company	tracking_number	raw_woo_payload
created_at	updated_at		

order_items

id	order_id	woo_order_item_id	inventory_item_id
line_number	sku	barcode	description
ordered_qty	ordered_uom	allocated_qty	picked_qty
unit_cost	unit_price	unit_cost_total	total_price
brand	status	created_at	updated_at

routes

id	route_name	route_date	status
start_address	end_address	total_stops	total_distance
estimated_duration	created_by	created_at	updated_at

route_stops

id	route_id	order_id	stop_number
customer_name	address_1	address_2	city
state	country	zip	phone
latitude	longitude	optimized_sequence	notes
created_at	updated_at		

import_jobs

id	file_name	import_type	total_rows
successful_rows	failed_rows	status	created_by
created_at	completed_at		

import_errors

id	import_job_id	row_number	sku
barcode	error_message	raw_row	created_at

16. API Surface Draft

Area	Scope
GET /health	Health check endpoint.
GET /api/items	List items with filters, search, pagination.
GET /api/items/{id}	Get one item.
POST /api/items	Create local item.
PATCH /api/items/{id}	Edit item.
POST /api/items/sync/woocommerce	Refresh products and variations from WooCommerce.
POST /api/items/{id}/remap	Remap local item to Woo product or variation.
GET /api/items/export	Export inventory summary CSV.
GET /api/items/export-by-location	Export inventory by location CSV.
POST /api/import/items	Upload inventory CSV for import preview.
POST /api/import/items/{job_id}/commit	Commit validated import job.
GET /api/import/items/{job_id}/errors.csv	Download failed rows.
GET /api/locations	List locations.
POST /api/locations/import	Import location CSV.
POST /api/receipts	Create direct receiving session.
GET /api/reports/received-inventory	Received inventory report CSV or JSON.
POST /api/cycle-counts	Submit cycle count.
POST /api/orders/sync/woocommerce	Sync WooCommerce orders.
GET /api/orders/open	Open Orders.

Area	Scope
POST /api/orders/{id}/allocate	Allocate order.
GET /api/orders/pick	Pick Orders list.
POST /api/orders/{id}/pick-scan	Scan SKU or barcode to pick item.
POST /api/orders/{id}/complete	Complete picked order and update WooCommerce.
GET /api/reports/fulfillment	Fulfillment report.
GET /api/reports/sku-orders	SKU or barcode order report.
POST /api/routes	Create route.
POST /api/routes/{id}/optimize	Optimize route.
GET /api/routes/{id}	Get route and stops.

17. Build Plan for Codex

1. Create repository and documentation only.
2. Add legacy WordPress plugin under legacy-wordpress-plugin/ as read-only reference.
3. Ask Codex to audit legacy plugin and document reusable logic. Do not modify the plugin.
4. Scaffold FastAPI backend, React frontend, PostgreSQL config, Alembic migrations, CORS, health check, and local setup.
5. Create database models and migrations.
6. Build Items module with local database data and fake/seed data first.
7. Build WooCommerce product and variation sync in read-only mode.
8. Add Refresh and Remap behavior.
9. Add inventory CSV import and export.
10. Add location import and location management.
11. Add direct receiving with location.
12. Add received inventory report.
13. Add cycle count.
14. Add WooCommerce order sync.
15. Add Open Orders, Allocate Orders, and Pick Orders.
16. Add fulfillment export and SKU/barcode order report.
17. Add route creation and route map placeholder.
18. Add route optimization provider interface.
19. Add Heroku deployment files when local MVP is stable.

18. Codex Starter Prompts

18.1 First Codex Prompt: Audit and Documentation

```
You are working on Pongo Inventory OS.

I have an existing WordPress/WooCommerce plugin in legacy-wordpress-plugin/pongo-inventory-os/.
Do not modify the legacy plugin.

The goal is not to convert the plugin line by line. The goal is to analyze it and create a
migration/rebuild plan for a standalone app.
```

```
New target stack:
- Backend: FastAPI
- Database: PostgreSQL
- ORM: SQLAlchemy
- Migrations: Alembic
- Frontend: React
- Deployment target: Heroku
- Integration: WooCommerce REST API

Analyze the legacy plugin and produce docs only.
Create:
- docs/LEGACY_PLUGIN_AUDIT.md
- docs/MIGRATION_MAP.md
- docs/REUSABLE_LOGIC.md
- docs/REBUILD_PLAN.md

In the audit, identify current modules, database tables, REST endpoints or AJAX handlers,
WooCommerce functions used, order workflow, inventory workflow, stock/audit logic, UI screens,
reusable ideas, and what must be rebuilt from scratch.

Do not build code yet. Do not scaffold the app yet. Do not change any existing plugin file.
```

18.2 Second Codex Prompt: Project Scaffold

```
Based on the documentation and the Pongo Inventory OS scope, create the new standalone project
structure.

Do not port all features yet.

Create:
- backend/
- frontend/
- docs/
- README.md

Backend should be FastAPI. Frontend should be React. Database should be PostgreSQL with
SQLAlchemy and Alembic.

Add only:
- health check endpoint
- environment config
- database connection setup
- placeholder frontend layout
- sidebar pages: Dashboard, Items, Inventory, Locations, Receiving, Orders, Cycle Count,
Reports, Routes, Settings

Do not connect WooCommerce yet. Do not implement stock changes yet. Do not touch the legacy
plugin.
```

18.3 Third Codex Prompt: Database Models

```
Implement the initial database models and Alembic migrations for Pongo Inventory OS.

Create models:
- inventory_items
- inventory_locations
- inventory_item_locations
- stock_movements
- receipts
- receipt_items
- orders
- order_items
- routes
```

```
- route_stops
- import_jobs
- import_errors

Use SQLAlchemy and PostgreSQL-compatible field types. Add model tests that verify basic record
creation. Do not build UI features yet. Do not connect WooCommerce yet.
```

18.4 Fourth Codex Prompt: Items Module MVP

```
Build the Items module using local database data only.

Do not connect WooCommerce yet.

Backend endpoints:
- GET /api/items
- GET /api/items/{id}
- POST /api/items
- PATCH /api/items/{id}
- GET /api/items/export

Frontend:
Build Admin -> Items page similar to the Zenventory Items page reference, but with Pongo
branding.

Main tab only for now.

Page should have search, category filter, active/inactive filter, Include Non Inventory
checkbox, Reset, Clear, Refresh placeholder, Remap placeholder, Export, Items table, and edit
action.

Table columns:
Image, SKU, Description, Category, UOM, Unit Cost, Sales Price, Recommended Retail Price,
Barcode, Brand, Manufacturer, Manufacturer Website, Warehouse, Inventory Location, Default
Location, In Stock, Allocated, Sellable, Under Par, On Order, Weight, Active status.

Acceptance criteria:
Items can be created, edited, searched, filtered, and exported as CSV.
```

19. Decisions Still Needed Before Building

- Confirm final stack: FastAPI plus React plus PostgreSQL is recommended.
- Confirm whether frontend will use plain React, Vite, Next.js, or another framework.
- Confirm authentication approach: simple staff login, JWT, or provider-based login.
- Confirm whether manual product creation should stay local-only in MVP or also create WooCommerce products.
- Confirm exact CSV formats for inventory import, location import, detailed inventory report, and route export.
- Confirm exact location data fields when uploaded.
- Confirm whether WooCommerce stock should be updated immediately after receiving/cycle count/order completion or queued for review at first.
- Confirm mapping source for Brand in WooCommerce.
- Confirm map/route provider choice and budget later.
- Confirm whether route optimization starts from warehouse and returns to warehouse, or only optimizes customer stops.